

AWS Expense Tracker Project

Presenter: Kaarthikeyan S S 23BCE2041

Vimal s 23BCE0345



Contents

01 Project Overview

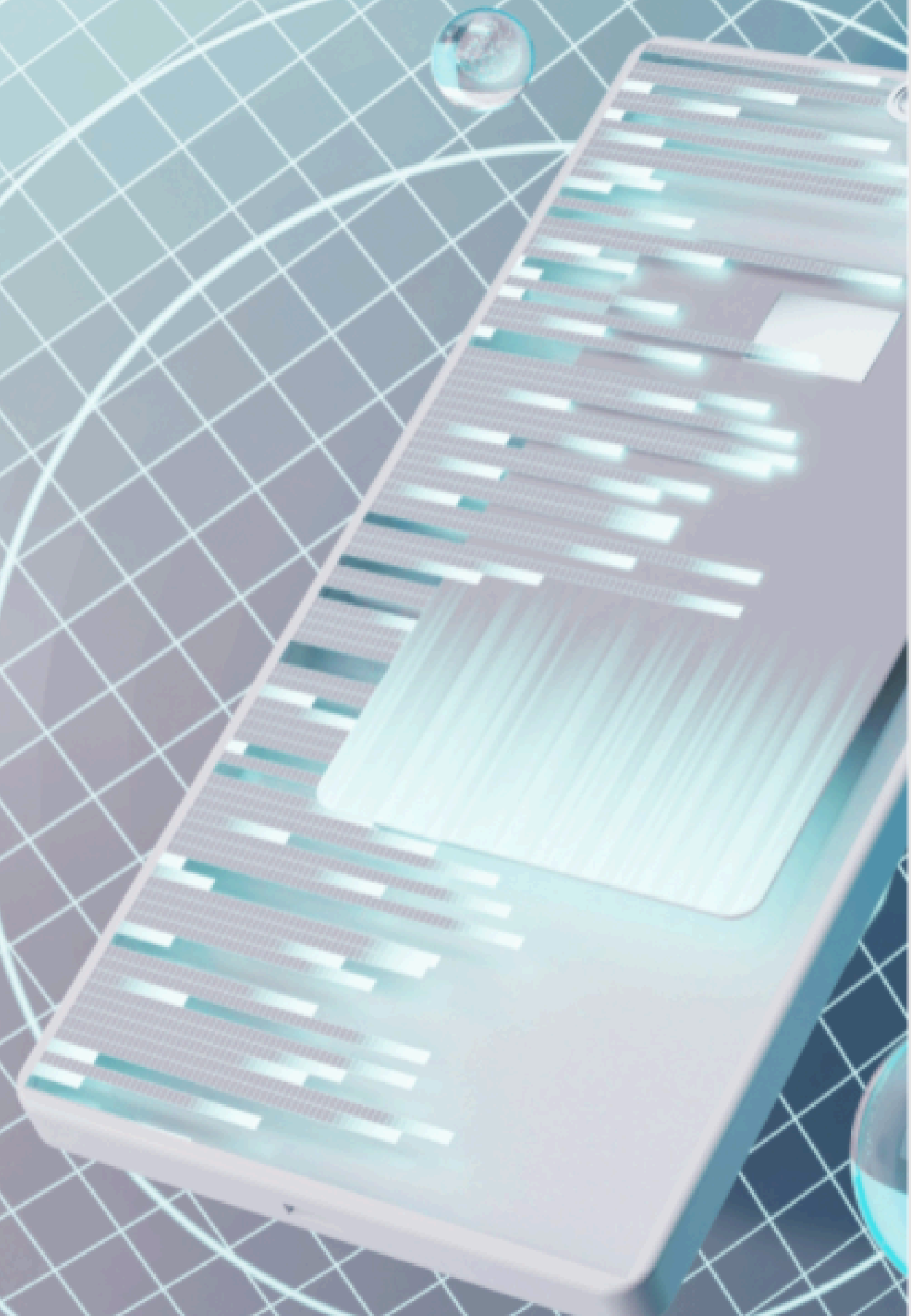
02 Architecture & Technologies

03 Implementation Details

04 Usage & Demo

01

Project Overview



Introduction

01

What is AWS Expense Tracker?

AWS Expense Tracker is a tool designed to monitor, analyze, and optimize cloud spending, helping users manage costs efficiently and gain clear insights into their AWS resource expenses.



02

Core AWS Services Used

The project utilizes core AWS services including AWS Cost Explorer for expense analysis, AWS Lambda for automation, Amazon S3 for data storage, and Amazon QuickSight for visualization and reporting.



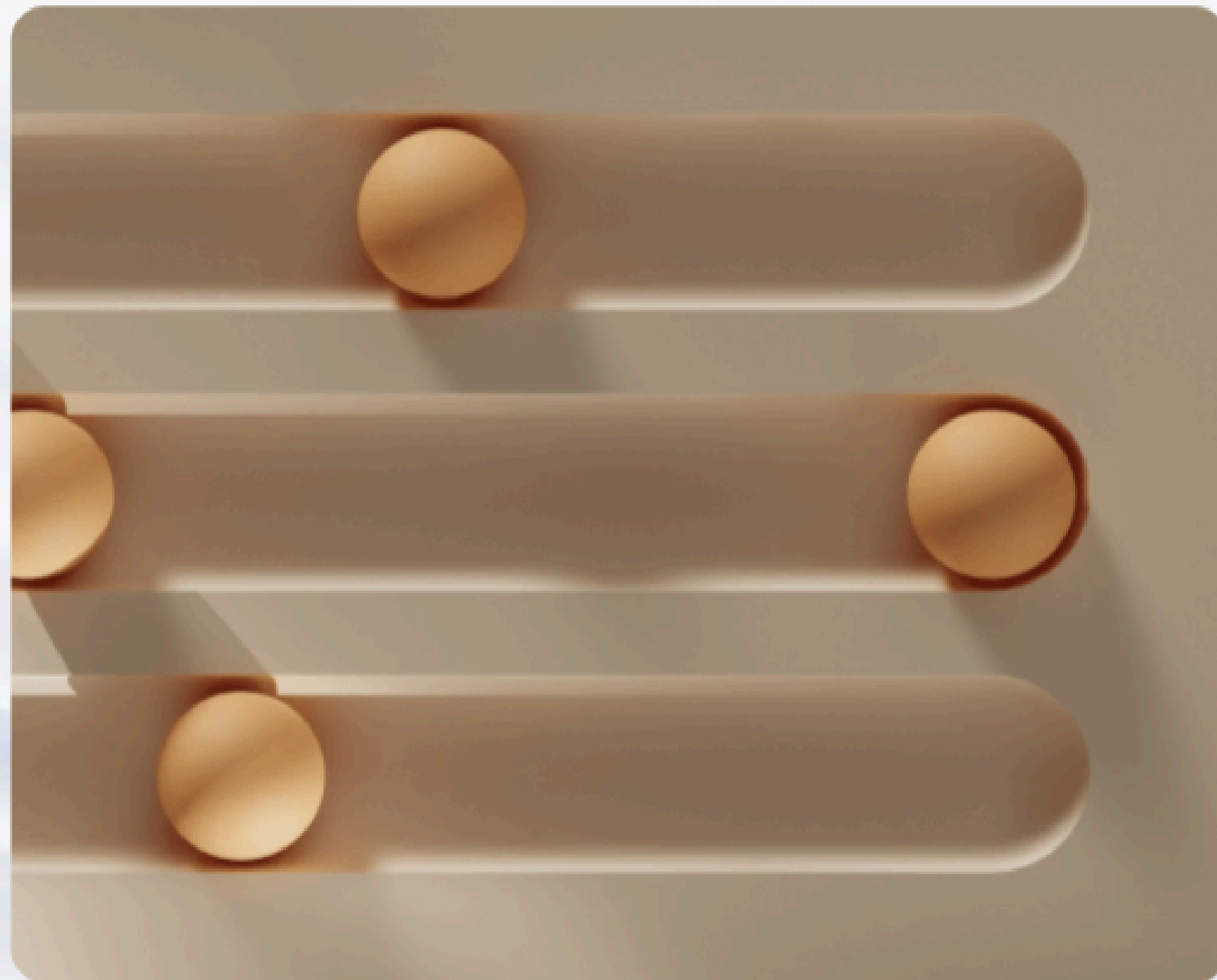
03

Project Objectives and Scope

The AWS Expense Tracker Project aims to monitor and optimize cloud spending by providing real-time expense tracking, cost analysis, and budgeting within defined AWS services and user groups.



Features



Expense Management via REST API

Enables seamless expense tracking and management through a secure REST API, allowing users to create, update, retrieve, and delete expense records efficiently in real-time.



DynamoDB for Data Storage

Utilizes DynamoDB for scalable, low-latency data storage, enabling efficient tracking and management of AWS expenses with high availability and seamless integration across services.



SNS Email Alerts for High Expenses

SNS Email Alerts notify users immediately when expenses exceed predefined thresholds, enabling proactive cost management and preventing unexpected high AWS charges through timely alerts and quick responses.

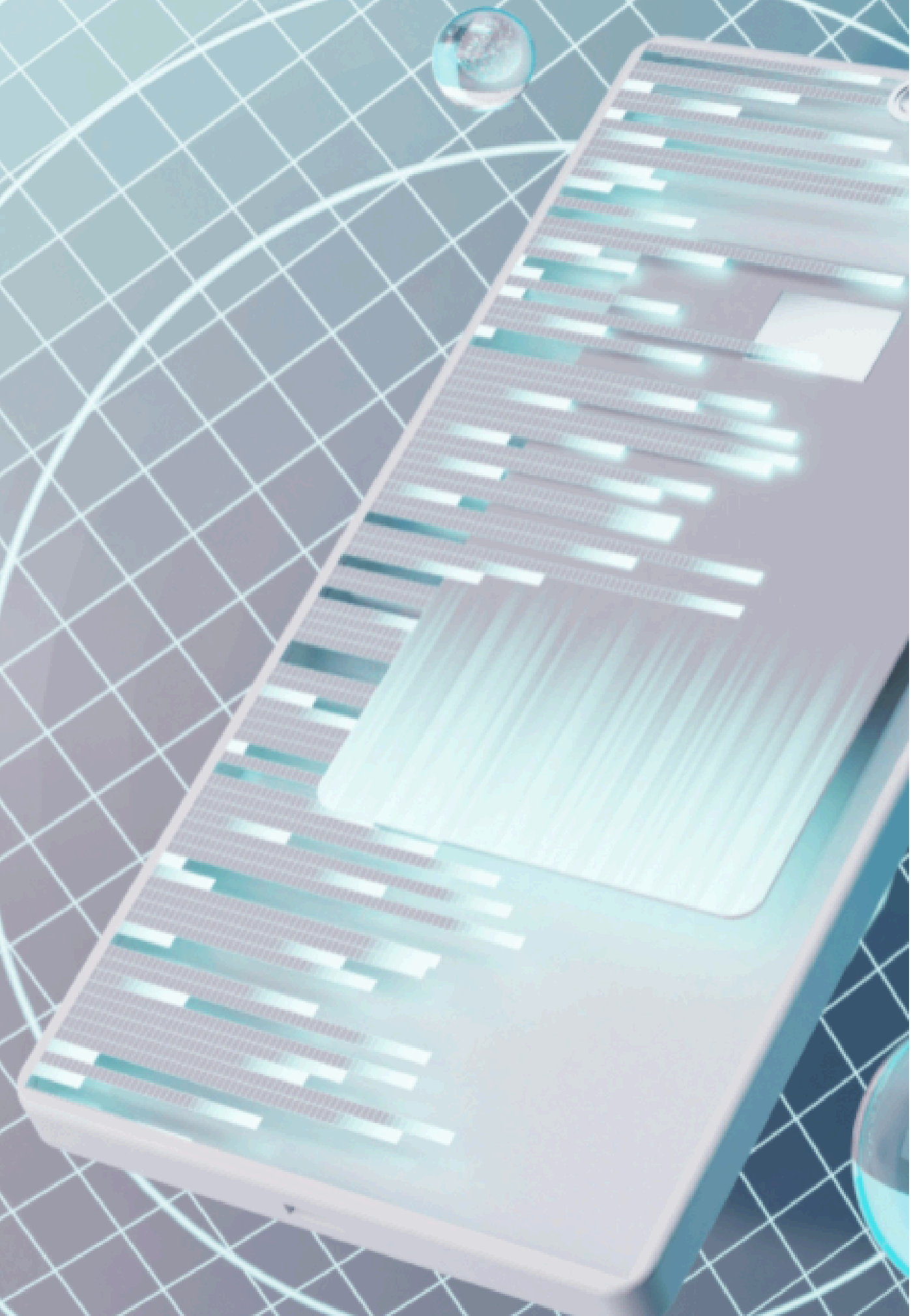


Serverless Deployment with Lambda & API Gateway

Utilizes AWS Lambda and API Gateway for scalable, cost-effective serverless deployment, enabling seamless API management, automatic scaling, and reduced infrastructure complexity in the AWS Expense Tracker.

02

Architecture & Technologies



AWS Services

Breakdown

✓ AWS Lambda Functions

AWS Lambda Functions automate backend processes by executing code in response to events, enabling scalable, cost-effective operations without managing servers, optimizing expense tracking workflows efficiently.

✓ API Gateway Integration

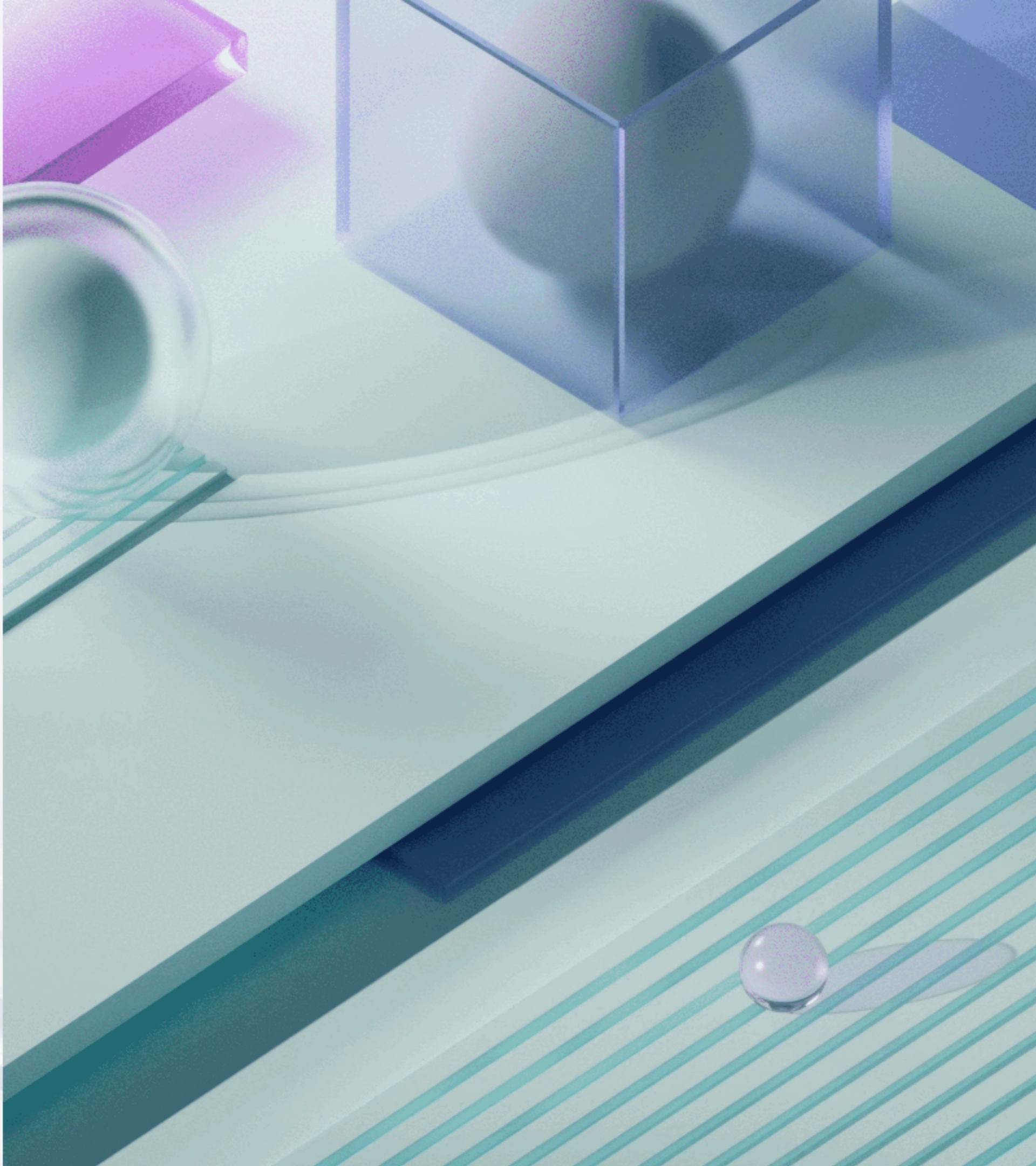
API Gateway serves as the entry point for client requests, enabling secure, scalable API management and seamless integration with AWS Lambda for backend processing in the expense tracker.

📊 DynamoDB Table Structure

DynamoDB uses a partition key for efficient query performance, with items storing expense details like user ID, timestamp, category, and amount, enabling fast, scalable expense tracking and retrieval.

☁️ SNS for Notifications

Amazon SNS enables real-time notifications for expense alerts and updates, ensuring timely communication. It supports multiple protocols, enhancing user engagement and system responsiveness in the AWS Expense Tracker Project.



Technology Stack

01

Python and Boto3 SDK

Python, combined with Boto3 SDK, enables efficient interaction with AWS services, automating expense data retrieval, processing, and management for seamless cloud cost tracking and analysis.

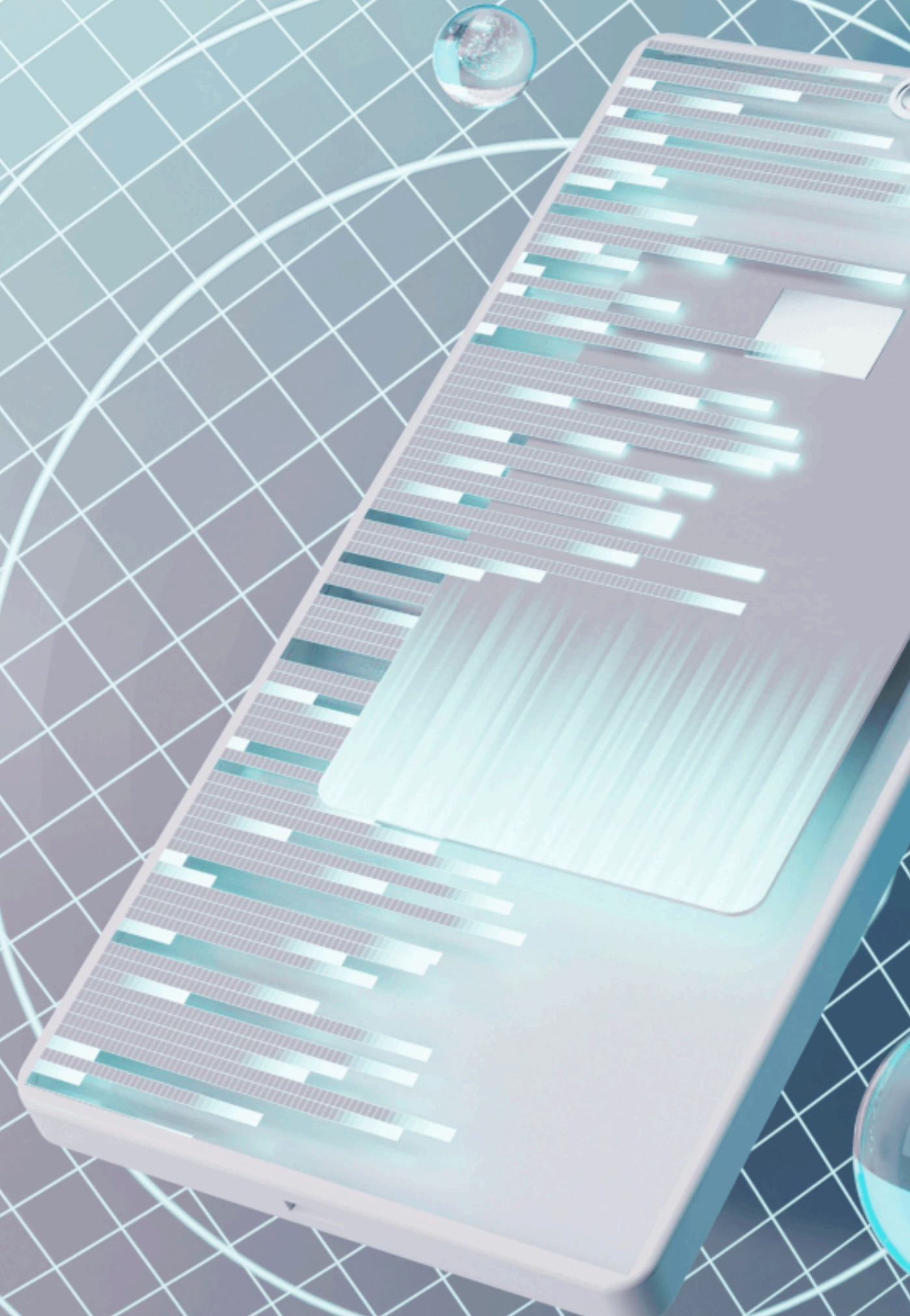
02

Serverless Architecture Benefits

Utilizing serverless architecture reduces operational overhead, enhances scalability, ensures cost-efficiency through pay-as-you-go, and accelerates deployment by abstracting infrastructure management.

03

Implementation Details



Project Structure

✓ Lambda Function Code (lambda_function.py)

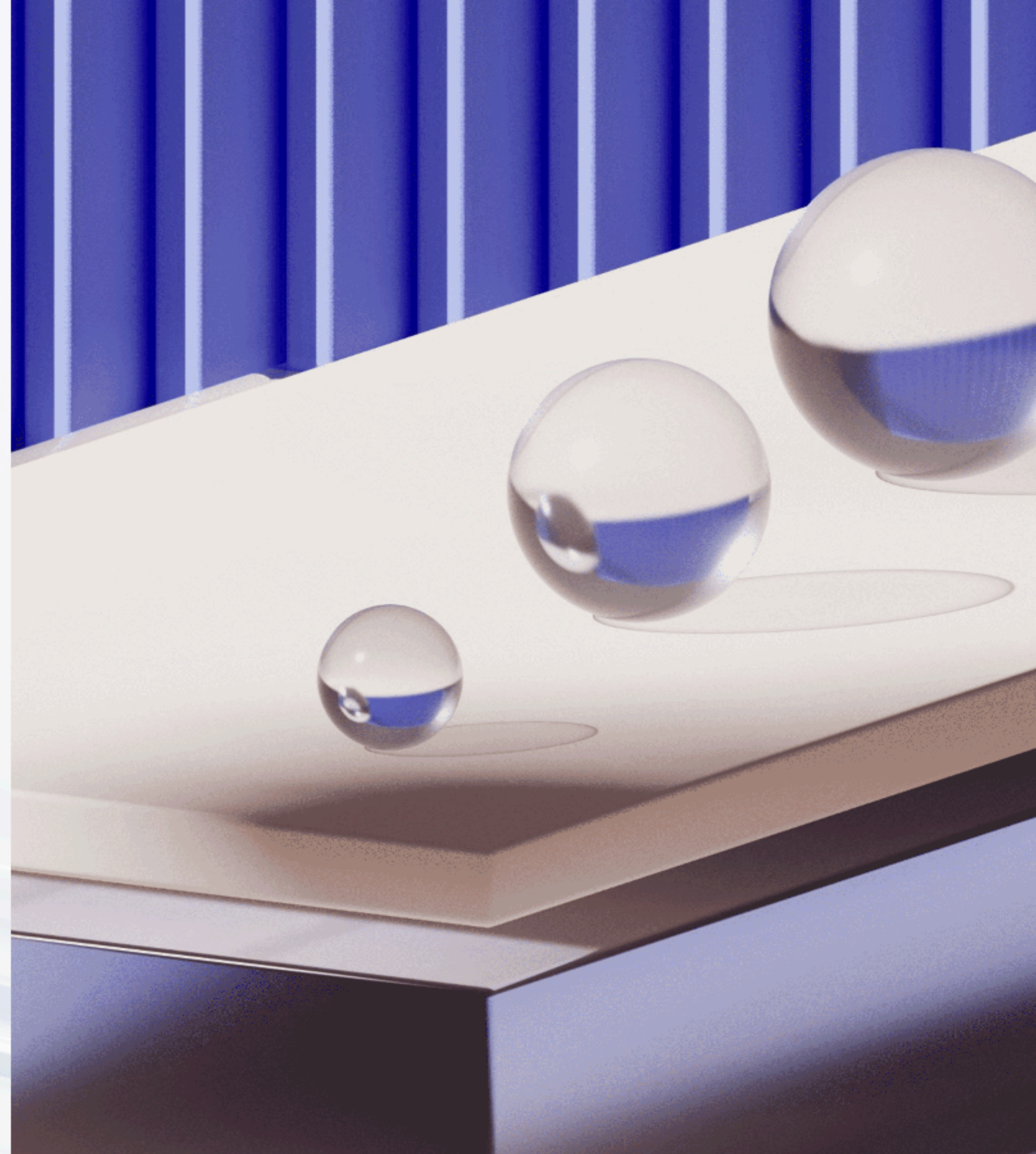
The lambda_function.py handles AWS expense data processing by fetching, parsing, and storing cost details. It integrates with AWS services like S3 and DynamoDB for efficient data management.

✓ Test Events and README

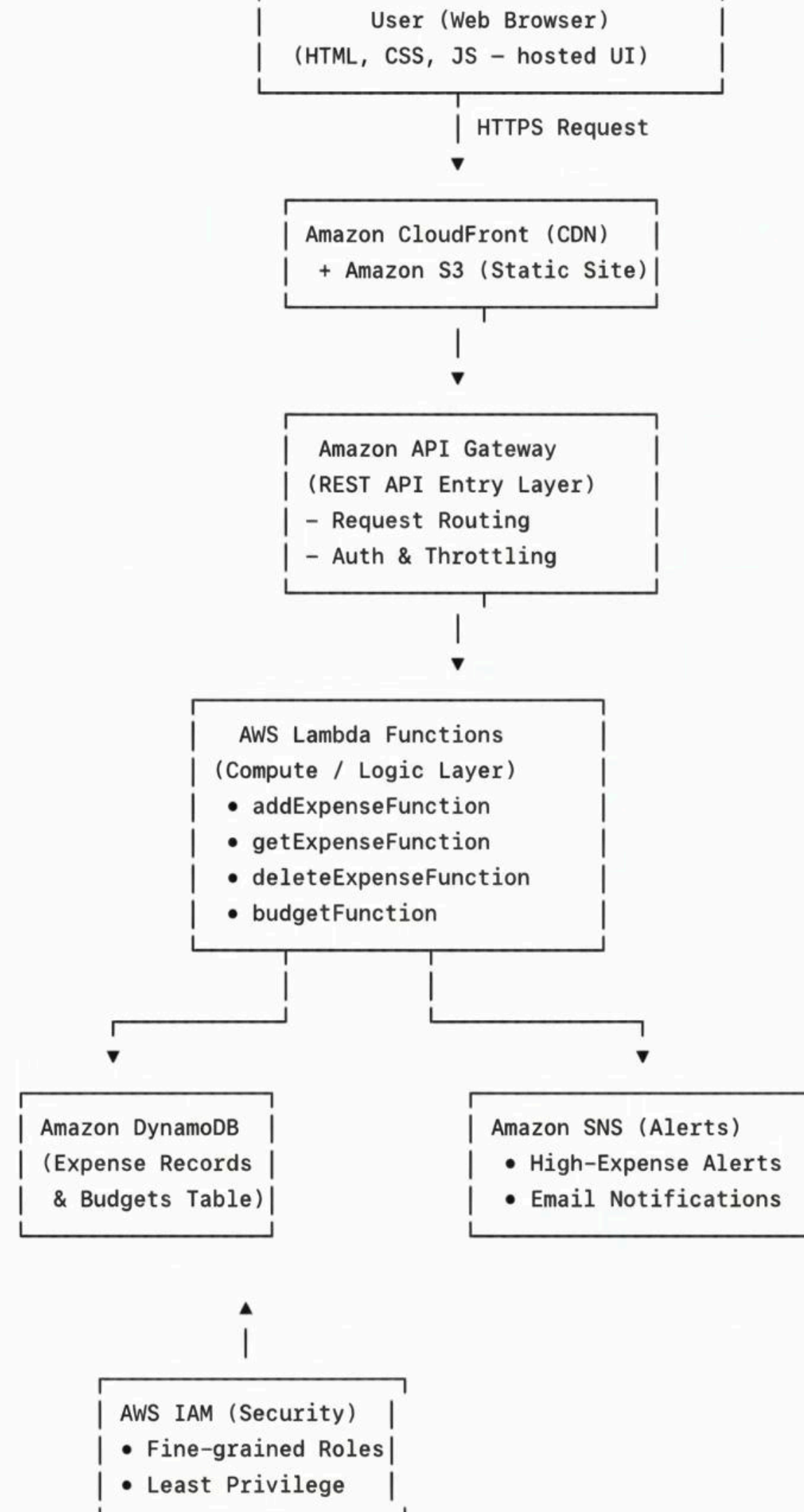
The project includes predefined test events to simulate different AWS expense scenarios, while the README offers clear setup instructions, usage guidelines, and troubleshooting tips for smooth implementation.

📁 Folder Organization

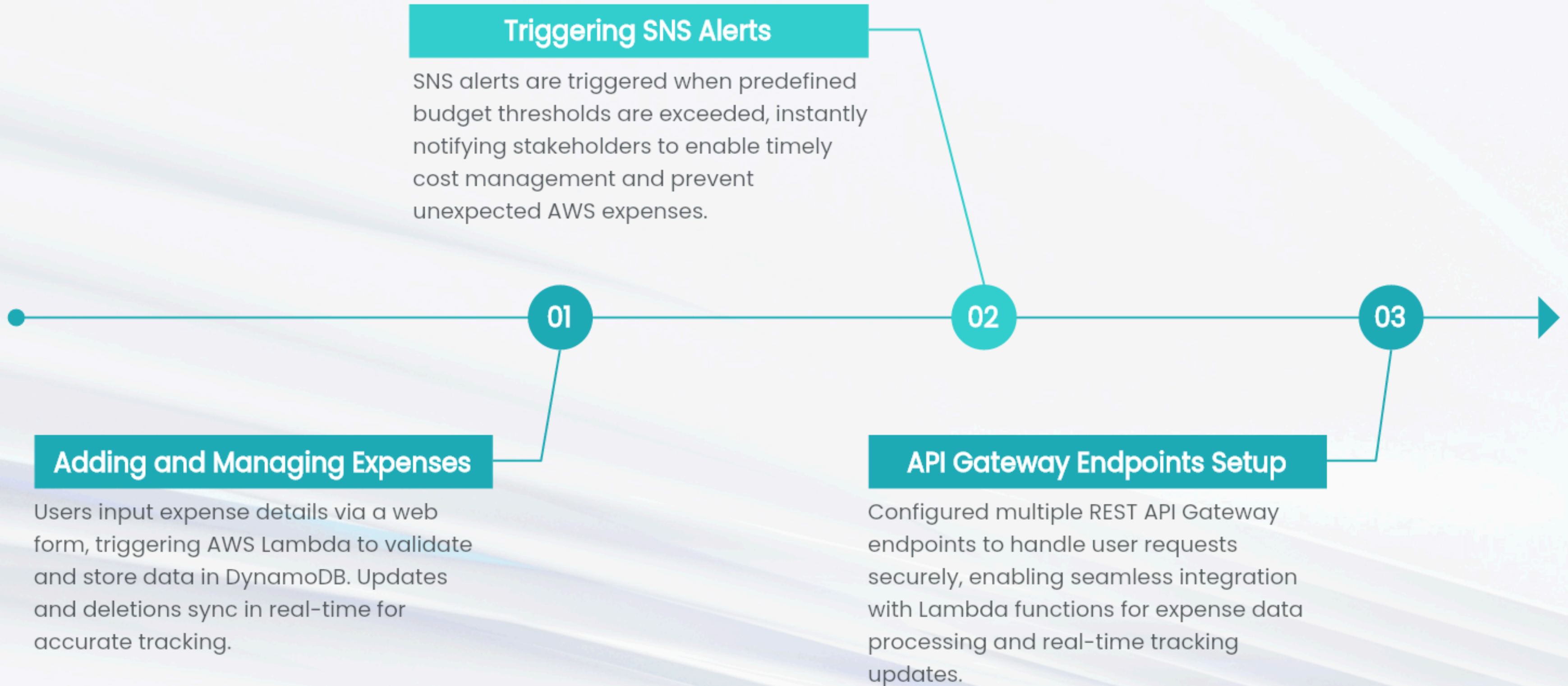
The project is organized into distinct folders separating backend, frontend, configurations, and utilities, enabling clear modular development and easier maintenance of the AWS Expense Tracker.



SYSTEM ARCHETECTURE

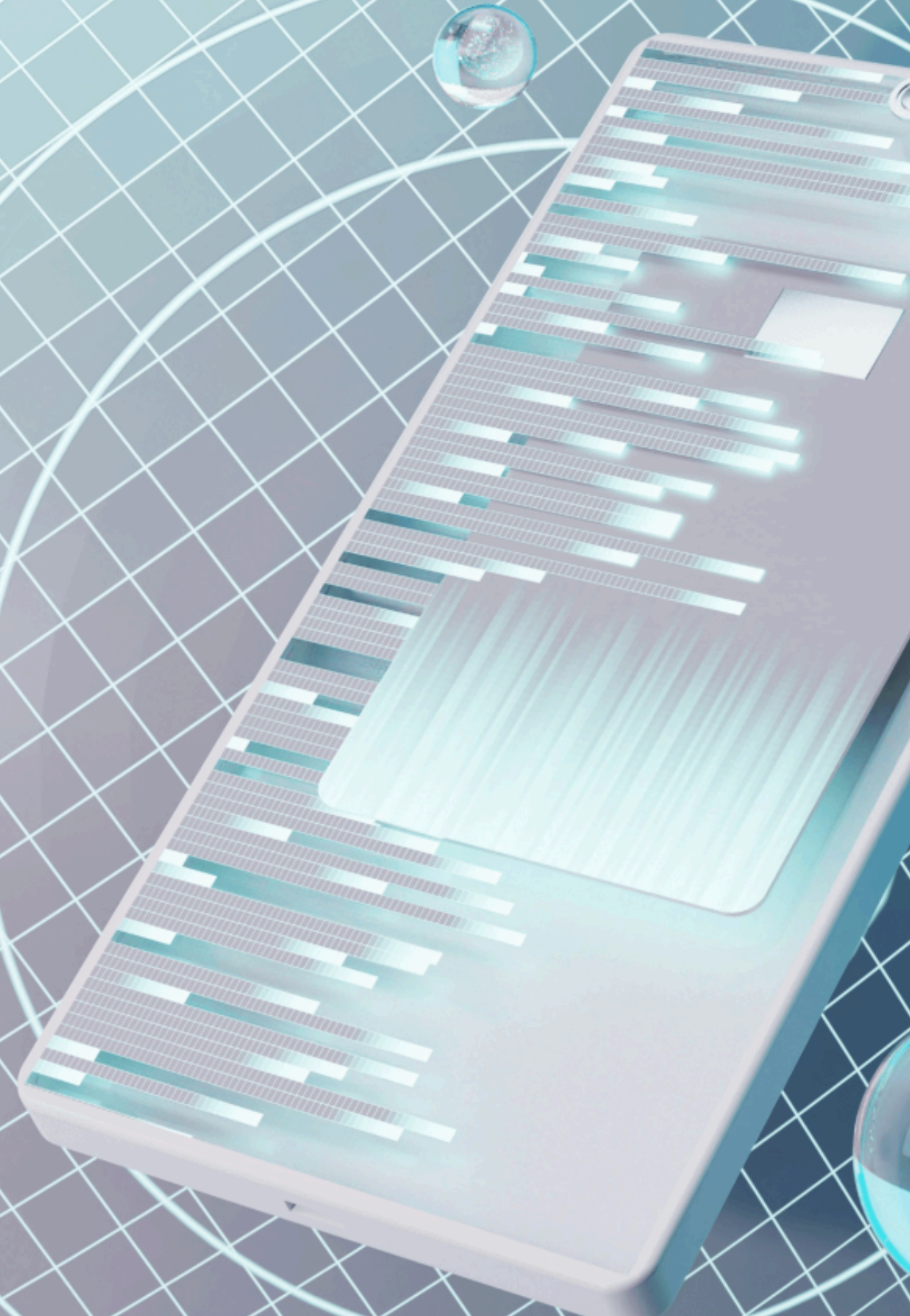


Workflow



04

Usage & Demo



Deployment Steps

01

Deploying Lambda Function

Configure AWS Lambda by packaging code, setting execution role, defining triggers, and deploying via AWS Console or CLI to automate expense data processing efficiently and securely.

02

Configuring API Gateway

Set up API Gateway by defining RESTful endpoints, enabling CORS, configuring request/response mappings, and integrating with Lambda functions to securely handle expense tracking operations and data transfers.

03

Setting Up SNS Email Subscription

Configured SNS to send alerts via email by subscribing user addresses, ensuring real-time notifications on expense updates and deployment statuses for effective monitoring and prompt response.

Testing and Interaction

Using Postman or HTML Form

Tested API endpoints using Postman for accurate request-response validation; interacted via HTML form to simulate user input, ensuring seamless data submission and real-time expense tracking functionality.

Sample SNS Email Alert Example

The SNS email alert demonstrates real-time cost notifications, highlighting monitored AWS service usage and triggering immediate action through concise, clear messaging for effective expense management.

Additional Resources

01

YouTube Video Demo Link

Includes a comprehensive YouTube video demo showcasing AWS Expense Tracker features, setup process, and practical usage, providing users with clear guidance and visual understanding for efficient implementation.



02

Documentation and Support

Comprehensive documentation guides seamless AWS Expense Tracker setup and usage. Responsive support forums and dedicated help channels ensure quick issue resolution and enhanced user experience.



Thank You

